TALLER GUIADO: SCENE BUILDER + JAVAFX

DSY1102 | Actividad 2.2 | Proyecto: EjemploSceneBuilder | Java 21 + Maven + JavaFX 21

Objetivo: construir desde cero una aplicación que recibe un nombre y muestra un saludo. La interfaz se diseña arrastrando controles en Scene Builder; el comportamiento se programa en IntelliJ IDEA.

Método de trabajo: realiza cada paso en orden, comprueba el resultado y recién después continúa. Esta guía utiliza el paquete cl.duoc.practicascene.

Módulo A. Crear el proyecto en IntelliJ IDEA

Paso 1. Crear un proyecto JavaFX

• En IntelliJ IDEA selecciona File > New > Project.

• Escoge JavaFX (si está disponible), Build system: Maven y JDK 21.

• Nombre del proyecto: EjemploSceneBuilder. Pulsa Create.

• Espera a que Maven descargue las dependencias y termine la indexación.

Comprobación: aparecen pom.xml, src/main/java, src/main/resources y, habitualmente, module-info.java.

Paso 2. Entender los archivos que crea IntelliJ

La plantilla JavaFX suele crear archivos de ejemplo como HelloApplication.java, HelloController.java y hello-view.fxml, además de module-info.java. Son demostraciones iniciales: en este taller crearemos App.java, SaludoController.java y saludo.fxml. Los nombres exactos de los archivos iniciales pueden variar según la versión de IntelliJ.

No borres todavía los archivos generados. Primero deja funcionando nuestra práctica; después podrás retirar los ejemplos que no uses.

Paso 3. Crear nuestros paquetes

• En src/main/java, crea el paquete cl.duoc.practicascene.

• Dentro de él crea el subpaquete controller.

• En src/main/resources, crea la ruta cl/duoc/practicascene (IntelliJ puede mostrarla compactada con puntos).

```
src/main/
  java/
    module-info.java
    cl/duoc/practicascene/
      App.java                 (se creará luego)
      controller/
        SaludoController.java  (se creará luego)
  resources/
    cl/duoc/practicascene/
      saludo.fxml              (se guardará desde Scene Builder)
```

Módulo B. Diseñar el formulario en Scene Builder

Paso 4. Crear un diseño nuevo

• Abre Scene Builder.

• Selecciona File > New.

• En Library, escribe VBox en el buscador.

• Arrastra VBox hacia el área central de diseño.

Explicación: VBox ordena sus controles verticalmente, uno debajo del otro.

Paso 5. Configurar el VBox

• Selecciona VBox en el panel Hierarchy.

• En Inspector > Layout configura Pref Width = 450 y Pref Height = 350.

• Configura Spacing = 15 y Alignment = CENTER.

• En Padding asigna 20 a Top, Right, Bottom y Left.

Comprobación: el área central tiene un contenedor de aproximadamente 450 x 350 píxeles.

Paso 6. Agregar el título

• En Library busca Label y arrástralo dentro del VBox.

• En Inspector > Properties cambia Text a Bienvenido a JavaFX.

• En la propiedad Font configura tamaño 20.

Paso 7. Agregar etiqueta y campo del nombre

• Arrastra un segundo Label y escribe Ingrese su nombre:.

• Arrastra un TextField debajo de ese Label.

• En Properties > Prompt Text escribe Escriba su nombre.

• Con el TextField seleccionado, abre Inspector > Code y escribe fx:id = txtNombre.

Importante: el identificador es txtNombre, no textNombre. Debe coincidir exactamente con el controlador.

Paso 8. Agregar el botón Saludar

• Arrastra un Button debajo del TextField.

• En Properties > Text escribe Saludar.

• En Inspector > Code > On Action escribe saludar (sin paréntesis).

Paso 9. Agregar la etiqueta de resultado

• Arrastra un tercer Label debajo del botón.

• En Properties > Text escribe Aquí aparecerá el saludo.

• En Inspector > Code escribe fx:id = lblResultado.

Comprobación: el VBox contiene cinco elementos en este orden: título, instrucción, TextField, Button y Label de resultado.

Módulo C. Guardar y vincular el diseño

Paso 10. Guardar saludo.fxml en la carpeta correcta

• En Scene Builder selecciona File > Save As.

• Navega hasta la carpeta de tu proyecto: src/main/resources/cl/duoc/practicascene/.

• Guarda el archivo con el nombre saludo.fxml.

• Vuelve a IntelliJ y verifica que aparece dentro de resources.

No lo guardes en una carpeta ajena al proyecto: App.java lo buscará en los recursos del paquete cl.duoc.practicascene.

Paso 11. Crear SaludoController.java

• En IntelliJ, haz clic derecho sobre cl.duoc.practicascene.controller.

• Selecciona New > Java Class y escribe SaludoController.

• Reemplaza el contenido por el siguiente código completo.

```
package cl.duoc.practicascene.controller;

import javafx.fxml.FXML;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;

public class SaludoController {

    @FXML
    private TextField txtNombre;

    @FXML
    private Label lblResultado;

    @FXML
    private void saludar() {
        String nombre = txtNombre.getText().trim();

        if (nombre.isEmpty()) {
            lblResultado.setText("Debe ingresar su nombre.");
            return;
        }

        lblResultado.setText(
            "¡Hola, " + nombre + "! Bienvenido a JavaFX."
        );
    }
}
```

Explicación: @FXML conecta las variables con los fx:id del diseño; trim() quita espacios exteriores; setText() modifica la etiqueta; el método saludar() responde al clic del botón.

Paso 12. Indicar el controlador en Scene Builder

• Vuelve a Scene Builder y abre el saludo.fxml guardado en resources.

• En el panel Document, despliega Controller.

• En Controller class escribe cl.duoc.practicascene.controller.SaludoController.

• Guarda con Ctrl + S.

Comprobación: Scene Builder agrega fx:controller al VBox. No necesitas escribirlo a mano.

Módulo D. Clase principal y configuración de Maven

Paso 13. Crear App.java

• En IntelliJ, crea la clase App dentro del paquete cl.duoc.practicascene.

• Copia el siguiente código completo.

```
package cl.duoc.practicascene;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.stage.Stage;
import java.io.IOException;

public class App extends Application {

    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader loader = new FXMLLoader(
            App.class.getResource("saludo.fxml")
        );

        Scene scene = new Scene(loader.load(), 450, 350);
        stage.setTitle("Práctica Scene Builder");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Explicación: FXMLLoader carga el diseño; Scene lo presenta; Stage representa la ventana principal.

Paso 14. Ajustar module-info.java

Si el proyecto tiene module-info.java, reemplaza su contenido por lo siguiente:

```
module cl.duoc.practicascene {
    requires javafx.controls;
    requires javafx.fxml;

    opens cl.duoc.practicascene to javafx.fxml;
    opens cl.duoc.practicascene.controller to javafx.fxml;

    exports cl.duoc.practicascene;
}
```

La instrucción opens permite que FXMLLoader acceda al controlador mediante reflexión. Esta guía asume que se mantiene el proyecto modular generado por IntelliJ.

Paso 15. Revisar las dependencias de pom.xml

En pom.xml comprueba que estén estas dos dependencias. Si ya existen, no las dupliques:

```
<dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-controls</artifactId>
    <version>21</version>
</dependency>
<dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-fxml</artifactId>
    <version>21</version>
</dependency>
```

Módulo D (continuación). Configurar la ejecución

Paso 16. Indicar la clase principal en Maven

Dentro del plugin org.openjfx:javafx-maven-plugin, busca <mainClass> y reemplaza su contenido por:

```
<mainClass>
    cl.duoc.practicascene/cl.duoc.practicascene.App
</mainClass>
```

El primer nombre es el módulo definido en module-info.java; el segundo es la clase principal con su paquete.

Paso 17. Confirmar el compilador Java 21

Comprueba que el plugin maven-compiler-plugin use Java 21:

```
<configuration>
    <source>21</source>
    <target>21</target>
</configuration>
```

No hace falta reemplazar todo el pom.xml si IntelliJ ya generó correctamente las dependencias y plugins.

Paso 18. Revisar el diseño generado

Abre saludo.fxml desde IntelliJ y verifica estas conexiones (Scene Builder las debe haber generado):

```
<VBox ...
      xmlns:fx="http://javafx.com/fxml/1"
      fx:controller="cl.duoc.practicascene.controller.SaludoController">

    ...
    <TextField fx:id="txtNombre" ... />
    <Button onAction="#saludar" text="Saludar" />
    <Label fx:id="lblResultado" ... />
    ...
</VBox>
```

El fragmento anterior es ilustrativo: no lo pegues como archivo completo. La estructura real puede contener atributos adicionales generados por Scene Builder.

Paso 19. Evitar confusiones con los archivos de ejemplo

La plantilla puede mantener HelloApplication, HelloController y hello-view.fxml. Para esta práctica no los ejecutaremos. Asegúrate de que Maven apunta a App y que App carga saludo.fxml. Una vez comprobado el funcionamiento, los archivos de ejemplo no usados pueden eliminarse si no tienen referencias pendientes.

Módulo E. Ejecutar, probar y comprender

Paso 20. Ejecutar el proyecto

• Guarda todos los archivos con Ctrl + S.

• En IntelliJ, recarga Maven si se solicita.

• En el panel Maven selecciona Plugins > javafx > javafx:run.

• También puedes ejecutar desde terminal: mvn clean javafx:run.

Resultado esperado: una ventana de 450 x 350 píxeles con el título, el campo de nombre, el botón y la etiqueta de resultado.

Paso 21. Prueba 1: nombre válido

• Escribe Ana en el campo de texto.

• Presiona Saludar.

• Comprueba que aparece: ¡Hola, Ana! Bienvenido a JavaFX.

Paso 22. Prueba 2: campo vacío

• Borra el nombre del campo.

• Presiona Saludar.

• Comprueba que aparece: Debe ingresar su nombre.

Paso 23. Prueba 3: espacios

• Escribe espacios antes y después de un nombre.

• Presiona Saludar.

• Comprueba que trim() elimina los espacios exteriores del saludo.

Paso 24. Cambiar el diseño sin modificar Java

• Abre saludo.fxml en Scene Builder.

• Cambia el texto del botón a Mostrar saludo.

• Modifica Spacing del VBox de 15 a 12.

• Guarda, ejecuta nuevamente y comprueba que saludar() sigue funcionando.

Esta prueba demuestra la separación entre diseño visual (FXML) y lógica de la aplicación (Java).

Paso 25. Preguntas para el cierre de la clase

• ¿Qué función cumple el VBox?

• ¿Cuál es la diferencia entre fx:id y Text?

• ¿Por qué el botón necesita On Action?

• ¿Qué hace fx:controller?

• ¿Qué sucede si el nombre está vacío?

• ¿Por qué App.java carga saludo.fxml desde resources?

Anexo 1. Código FXML de referencia

Este es un ejemplo completo del resultado esperado. Se incluye para comparar, no para sustituir la construcción visual en Scene Builder. La versión generada por tu instalación puede ordenar los atributos de otra forma.

```
<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.VBox?>
<?import javafx.scene.text.Font?>

<VBox alignment="CENTER" prefHeight="350.0"
      prefWidth="450.0" spacing="15.0"
      xmlns="http://javafx.com/javafx/21"
      xmlns:fx="http://javafx.com/fxml/1"
      fx:controller="cl.duoc.practicascene.controller.SaludoController">
    <padding>
        <Insets top="20.0" right="20.0"
                bottom="20.0" left="20.0" />
    </padding>
    <children>
        <Label text="Bienvenido a JavaFX">
            <font><Font size="20.0" /></font>
        </Label>
        <Label text="Ingrese su nombre:" />
        <TextField fx:id="txtNombre"
                   promptText="Escriba su nombre" />
        <Button text="Saludar" onAction="#saludar" />
        <Label fx:id="lblResultado"
               text="Aquí aparecerá el saludo" />
    </children>
</VBox>
```

Nota: Scene Builder 26 puede escribir un namespace /26 aunque el proyecto ejecute JavaFX 21. Para este ejercicio usamos controles compatibles con JavaFX 21; conviene mantener consistencia de versiones si luego se utilizan propiedades nuevas.

Anexo 2. Lista de verificación antes de entregar

- Existe App.java dentro de cl.duoc.practicascene.
- Existe SaludoController.java dentro de controller.
- Existe saludo.fxml dentro de resources/cl/duoc/practicascene.
- Los fx:id son txtNombre y lblResultado.
- El botón tiene On Action = saludar.
- El VBox tiene asignado SaludoController como controlador.
- module-info.java y mainClass coinciden con el paquete real.
- La aplicación supera las tres pruebas de entrada.

Anexo 3. Errores frecuentes y cómo prevenirlos

No controller specified

Causa: el archivo FXML no tiene fx:controller. Solución: Document > Controller > Controller class en Scene Builder; guardar el archivo.

NullPointerException al leer txtNombre

Causa probable: fx:id no coincide con el atributo del controlador. Verificar txtNombre exactamente, respetando mayúsculas y minúsculas.

Error al abrir FXML: XML declaration ... line 2

Causa: existe una línea vacía o caracteres antes de <?xml. Solución: la declaración XML debe comenzar en la primera posición del archivo.

Location is not set / FXML no encontrado

Causa: saludo.fxml está en otra carpeta. Solución: ubicarlo en resources/cl/duoc/practicascene/ para usar App.class.getResource("saludo.fxml").

Maven sigue ejecutando HelloApplication

Causa: mainClass conserva la clase de ejemplo. Solución: cambiarla a cl.duoc.practicascene/cl.duoc.practicascene.App.

JavaFX 26 / runtime 21

Scene Builder puede guardar un namespace de versión 26 mientras Maven ejecuta JavaFX 21. No siempre impide abrir el formulario; revisar compatibilidad si se usan propiedades de una versión posterior.

Siguiente actividad

Aplicaremos este mismo procedimiento al proyecto GestionMaquinariasFX: abrir registro.fxml en Scene Builder, organizar los controles, mantener los fx:id y eventos existentes y mejorar la distribución de la TableView.